SpectrumKV: Per-Token Mixed-Precision KV Cache Transfer for Prefill–Decode Disaggregated LLM Serving

Pengju Yang
Jilin University
yangsteve1223@github

May 2026

Abstract

Prefill–decode (PD) disaggregation decouples prompt processing from token generation, but it also turns the key–value (KV) cache into a network payload. Existing PD-side KV reduction methods are mostly binary: selected tokens are transmitted at full precision and the rest are not transmitted. This paper argues that binary selection leaves a useful design space unused. SpectrumKV assigns a precision level to each token instead: attention sinks and other high-importance tokens are protected at FP16, medium-importance tokens are sent at INT8, and low-importance tokens are sent at INT4 when the model can tolerate it.

The main practical complication is that INT4 tolerance is model-dependent. Qwen2.5-7B catastrophically fails under INT4 KV quantization, while Mistral-7B and Gemma-2-9B remain stable. SpectrumKV therefore runs a lightweight deployment-time probe: three aggressive NIAH trials under a 3-tier policy. Models that pass use FP16+INT8+INT4; models that fail fall back to FP16+INT8.

Across Qwen2.5-7B-Instruct, Mistral-7B-Instruct-v0.3, and Gemma-2-9B-it, SpectrumKV improves quality at the same transfer budget. At a 50% normalized KV budget on WikiText-2, SpectrumKV changes perplexity by +1.97%, −0.06%, and −0.44%, respectively, compared with PDTrim's +25.85%, +22.07%, and +35.63%. On NIAH retrieval at 4096 tokens, the adaptive policy reaches 52.6% on Qwen at the aggressive $b = 0.3$ budget versus 26.3% for PDTrim, and reaches 100% by $b = 0.5$; Mistral and Gemma preserve retrieval under the 3-tier policy. End-to-end GPU timing of the transfer path shows 50–62% TTFT reductions at $b = 0.5$. These results suggest that PD KV transfer should be treated as a precision-allocation problem, not only as token pruning.

Keywords: LLM serving, KV cache, prefill–decode disaggregation, mixed precision, KV quantization, communication compression, attention sinks.

Artifacts. The project repository (`https://github.com/YangSteve1223/kvcache-lab`) contains: (1) all Python experiment scripts for locality characterization, PPL sweeps, NIAH retrieval, GPU timing, and quantization error analysis; (2) raw JSON logs for every table and figure; (3) table and figure regeneration code; (4) model configuration files with pre-trained model download links and chat templates; (5) the exact decay-rate, budget, and first-ratio hyperparameters used in each run.

1 Introduction

PD disaggregation is becoming a standard serving pattern for large language models because prefill and decode stress different resources. Prefill is parallel and compute-heavy; decode is sequential and latency-sensitive. Systems such as Splitwise, DistServe, and Mooncake therefore split the two phases across different workers so that each phase can be scheduled and scaled independently[15;16;21]. The price of this separation is that the KV cache produced during prefill must be made available to decode. In long-context serving, that cache is large enough for transfer bandwidth and transfer latency to become first-order concerns.

A common response is to transmit fewer tokens. StreamingLLM identifies attention sinks and shows that the beginning of the sequence can remain important for stable generation [17]; H_2O, SnapKV, PyramidKV, and related methods select or compress tokens according to attention-derived importance [1;9;20;22]. PDTrim adapts this keep/drop view to PD transfer by retaining selected first/last regions and pruning the rest [19]. These methods are valuable, but the binary decision is unnecessarily harsh in the PD setting. The situation parallels operating-system memory management: just as a virtual memory system need not evict a cold page entirely—it can compress or swap it to a slower tier while preserving a stub for fast reclamation—a PD transfer system need not drop a cold KV token entirely. A reduced-precision copy still carries useful information, whereas a dropped token carries none. Must the choice always be binary—retain fully or discard entirely—or can a token be partially preserved?

SpectrumKV takes a different view: every token can be represented on a precision spectrum. The policy assigns FP16 to tokens whose perturbation would be most harmful, INT8 to medium-importance tokens, and INT4 to low-importance tokens when the model tolerates INT4. This is not the same as uniform quantization. Uniform INT8 treats all positions equally; SpectrumKV reserves high fidelity for attention sinks, recent tokens, or other positions with high estimated contribution to decode. It is also not the same as pruning. The low tier still transmits information, which matters especially for retrieval tasks where the needle may appear outside a selected window.

The strongest empirical lesson from this project is that INT4 is not a universally safe low tier. Qwen2.5-7B is local-dominant and extremely sensitive to INT4 KV perturbations: a balanced 3-tier assignment at $b = 0.5$ increases PPL by +7506.84%. By contrast, Mistral-7B and Gemma-2-9B tolerate INT4 in low-importance positions with PPL changes under 6% and 100% NIAH accuracy at $b = 0.3$. SpectrumKV therefore includes a small probe that decides whether to enable INT4 for each model.

This paper makes the following contributions.

1. We formulate PD KV communication reduction as *per-token precision allocation* under a normalized transfer budget, rather than as binary keep/drop pruning.
2. We propose SpectrumKV, a mixed-precision policy with sink protection, importance-ranked tier assignment, and a lightweight INT4 tolerance probe.
3. We provide GPU-verified results on Qwen2.5-7B-Instruct, Mistral-7B-Instruct-v0.3, and Gemma-2-9B-it, covering PPL, NIAH retrieval, TTFT/TPS timing, context length, multi-task robustness, and quantization error.
4. We identify a model-specific INT4 failure mode that is not predicted by per-vector cosine similarity of quantized KV vectors but is explained by softmax amplification under low-entropy attention. This motivates empirical probing before enabling aggressive KV quantization.
5. We provide a systematic ablation isolating sink protection, importance ranking, and the INT4 tolerance probe, and we extend the theoretical framework with softmax amplification bounds and mixed-precision error propagation analysis.

FloatBarrier

2 Problem Formulation

The preceding argument rests on a claim that per-token precision allocation can outperform binary keep/drop decisions. We now formalize this claim: we define the transfer budget, derive error bounds that expose the asymmetry between approximation and deletion, and show why an importance-sorted assignment follows naturally from the structure of the problem.

2.1 KV transfer budget

Consider a decoder-only Transformer with L layers, H_{kv} key/value heads, head dimension d_h, and prompt length n. The full-precision prefill KV payload contains keys and values for every layer and token:

$$\text{Bytes}_{\text{FP16}}(n) = 2\, L\, H_{kv}\, d_h\, n\, s_{16}, \tag{1}$$

where $s_{16} = 2$ bytes for FP16 and the leading factor of two accounts for keys and values.

Let each token j be assigned a tier $q_j \in \{16, 8, 4\}$ with normalized per-element cost

$$c(16) = 1, \qquad c(8) = \frac{1}{2}, \qquad c(4) = \frac{1}{4}. \tag{2}$$

The normalized transfer budget is

$$b(\pi) = \frac{1}{n} \sum_{j=1}^{n} c(q_j), \tag{3}$$

where π denotes the precision assignment. A budget of $b = 0.5$ therefore corresponds to sending all tokens at INT8, or to any mixture with the same average byte cost. This point is important for interpreting the experiments: at $b = 0.5$, several policies collapse to the same all-INT8 behavior because INT8 is nearly lossless in our measurements.

Selection methods can be written in the same budget language by assigning $c = 1$ to retained FP16 tokens and $c = 0$ to dropped tokens. The distinction is semantic, not just arithmetic: a dropped token has no representation at decode time, while a low-precision token still contributes an approximate key/value vector.

2.2 Precision assignment as weighted approximation

At a decode step, one attention head produces

$$y = \sum_{j=1}^{n} p_j v_j, \tag{4}$$

where p_j is the attention probability and v_j is the value vector. If token j is quantized to tier q_j, let $\tilde{v}_j = v_j + e_j$ and assume $\|e_j\| \le \delta(q_j)$. Then

$$\|y - \tilde{y}\| = \left\| \sum_{j=1}^{n} p_j (v_j - \tilde{v}_j) \right\| \le \sum_{j=1}^{n} p_j \delta(q_j) = \sum_{t \in \{16,8,4\}} \delta(t) \sum_{j : q_j = t} p_j. \tag{5}$$

This bound is simple but useful: the error contribution of a tier scales with the *attention mass assigned to that tier*, not merely with the number of tokens in the tier. It motivates sending high-attention tokens at high precision and low-attention tokens at lower precision.

Dropping is the limiting case with no approximation. Let R be the removed set, L the retained set, and $\epsilon = \sum_{j \in R} p_j$. If the runtime renormalizes attention over L, the local output is

$$\hat{y} = \sum_{j \in L} \frac{p_j}{1 - \epsilon} v_j. \tag{6}$$

Lemma 1 (Tail-mass perturbation)**.** *If $\|v_j\| \le V$ for all j and $\epsilon < 1$, then*

$$\|y - \hat{y}\| \le 2\epsilon V. \tag{7}$$

Proof. Write $y = (1 - \epsilon)\mu_L + \epsilon \mu_R$, where μ_L and μ_R are convex combinations of retained and removed values. The renormalized output is $\hat{y} = \mu_L$. Thus $y - \hat{y} = \epsilon(\mu_R - \mu_L)$ and $\|y - \hat{y}\| \le \epsilon(\|\mu_R\| + \|\mu_L\|) \le 2\epsilon V$. □

The bound explains why pruning can work when the removed tail has low mass, but it also exposes the retrieval problem: if the removed token is the one containing the answer, its contribution is exactly zero. Mixed precision changes the failure mode from information deletion to approximation error.

2.3 Why a sorted assignment is natural

Suppose each token has an importance score $I_j \geq 0$ that approximates its contribution to downstream error, and each tier has cost c_t and perturbation level δ_t with

$$c_{16} > c_8 > c_4, \qquad \delta_{16} < \delta_8 < \delta_4. \tag{8}$$

A first-order objective is

$$\min_{q_1,\ldots,q_n} \sum_{j=1}^{n} I_j \delta(q_j) \quad \text{s.t.} \quad \frac{1}{n} \sum_{j=1}^{n} c(q_j) \leq b. \tag{9}$$

Proposition 1 (Exchange argument)**.** *For any fixed number of tokens assigned to each tier, an optimal assignment gives no lower-precision tier to a token with higher importance while giving a higher-precision tier to a lower-importance token.*

Proof. Consider two tokens a, b with $I_a \geq I_b$ and two tiers r, s with $\delta_r < \delta_s$. Assigning a to s and b to r has weighted error $I_a\delta_s + I_b\delta_r$. Swapping them gives $I_a\delta_r + I_b\delta_s$. The improvement is $(I_a - I_b)(\delta_s - \delta_r) \geq 0$. Repeated swaps yield an importance-sorted assignment. □

This does not claim that a single scalar score perfectly predicts quality. It only justifies the core design: once a score is chosen, high-importance tokens should receive at least as much precision as low-importance tokens.

2.4 Key quantization and softmax amplification

Value perturbation is not the only issue. Quantizing keys perturbs attention logits. Let $z_j = q^\top k_j/\sqrt{d}$ and $\tilde{z}_j = z_j + \eta_j$ with $|\eta_j| \leq \eta$. Then the softmax probabilities obey

$$e^{-2\eta} p_j \leq \tilde{p}_j \leq e^{2\eta} p_j. \tag{10}$$

The bound is loose, but it captures the qualitative risk: if attention is low-entropy and dominated by a few positions, small key perturbations around those positions can cause large relative changes in the normalized distribution. This mechanism explains Qwen's behavior: its local-dominant pattern makes INT4 unsafe even though its average INT4 key cosine similarity is high.

2.5 Softmax amplification under low-entropy attention

The element-wise bound above does not account for how the *distribution shape* interacts with perturbation. We now make this interaction explicit.

Definition 1 (Attention entropy)**.** For attention probabilities $p = (p_1, \ldots, p_n)$, the attention entropy is $H(p) = -\sum_{j=1}^{n} p_j \log p_j$, and the sparsity measure is $\|p\|_1^2/\|p\|_2^2$, which equals the inverse participation ratio.

When attention is concentrated (low entropy), the logits of the dominant tokens are large relative to the rest. A small perturbation η_j on a dominant token's key shifts its logit by η_j, but because the softmax denominator is dominated by a few large logits, the relative probability change on that token is amplified.

Lemma 2 (Softmax amplification bound)**.** *Let $p_j = \mathrm{softmax}(z)_j$ and $\tilde{p}_j = \mathrm{softmax}(\tilde{z})_j$ where $\tilde{z}_j = z_j + \eta_j$ with $\|\eta\|_\infty \le \eta$. Let $S = \{j : z_j \ge \max_k z_k - \log(1/\epsilon)\}$ be the set of tokens whose logits are within $\log(1/\epsilon)$ of the maximum. Then for any $j \in S$:*

$$\frac{|\tilde{p}_j - p_j|}{p_j} \le e^{2\eta} - 1 + \frac{|S| \cdot e^{2\eta} - |S|}{1 + |S| \cdot e^{-2\eta}}. \tag{11}$$

When $|S|$ is small (low-entropy regime) and η is moderate, the bound scales roughly as $e^{2\eta}/|S|$, meaning fewer dominant tokens yield larger relative perturbations.

Proof. Write $Z = \sum_k e^{z_k}$ and $\tilde{Z} = \sum_k e^{\tilde{z}_k}$. For $j \in S$:

$$\frac{\tilde{p}_j}{p_j} = \frac{e^{\tilde{z}_j}/\tilde{Z}}{e^{z_j}/Z} = e^{\eta_j} \cdot \frac{Z}{\tilde{Z}}.$$

Since $e^{z_j - \eta} \le e^{\tilde{z}_j} \le e^{z_j + \eta}$, we have $Z \cdot e^{-\eta|S|} \cdot (n - |S|)\epsilon \le \tilde{Z} \le Z \cdot e^{\eta|S|} \cdot n$. The tightest bound arises when the partition is sharp: tokens in S carry nearly all the mass. Then $\tilde{Z} \approx Z \cdot e^{\pm\eta} \cdot |S|$, giving the stated result. The full derivation tracks the contribution of non-dominant tokens, which adds the correction term. □

The practical implication is that models with local-dominant attention patterns (small $|S|$, low entropy) are more vulnerable to key quantization perturbations than models with broader sink or hybrid patterns. This explains why Qwen, despite having higher per-vector cosine similarity under INT4, suffers catastrophically: its attention mass concentrates on a few nearby tokens, so even small key perturbations produce large relative probability shifts.

Conjecture 1 (Entropy-dependent INT4 tolerance)**.** *For a Transformer with average per-head attention entropy $\bar{H}$, if $\bar{H} < H^*$ for some threshold H^*, then INT4 KV quantization causes non-linear quality degradation that cannot be predicted from per-vector cosine similarity alone. The threshold H^* depends on the model's depth and residual stream norm but appears to be approximately $H^* \approx \log n - 2$ for sequence length n in our experiments.*

We state this as a conjecture rather than a theorem because the precise threshold H^* depends on layer-wise interactions that we have not fully characterized. The empirical evidence (Section 7.4 and Section 8) is consistent with this claim.

2.6 Mixed-precision error propagation across layers

The per-layer bound in Eq. (5) treats each layer independently. In practice, KV errors propagate through the residual stream. We analyze this propagation.

Consider an L-layer Transformer. Let the output of layer l be $x^{(l)} = x^{(l-1)} + a^{(l)}$, where $a^{(l)}$ is the attention output. If the KV cache at layer l has perturbation δ_l (from Eq. (5)), the perturbed attention output satisfies $\|\tilde{a}^{(l)} - a^{(l)}\| \le \delta_l$. Then:

Proposition 2 (Layer-wise error propagation)**.** *If each layer l has attention perturbation δ_l and the layer norm at layer l has Lipschitz constant Λ_l, then the output perturbation after L layers satisfies:*

$$\|\tilde{x}^{(L)} - x^{(L)}\| \le \sum_{l=1}^{L} \delta_l \cdot \prod_{k=l+1}^{L} (1 + \Lambda_k \cdot \alpha_k), \tag{12}$$

where α_k is the attention amplification factor at layer k, bounded by $e^{2\eta_k}/|S_k|$ from Lemma 2.

Proof. By induction. At layer 1: $\|\tilde{x}^{(1)} - x^{(1)}\| = \|\tilde{a}^{(1)} - a^{(1)}\| \le \delta_1$. At layer $l+1$, the input perturbation $\Delta^{(l)}$ propagates through layer norm (bounded by Λ_{l+1}) and attention (amplified by α_{l+1}):

$$\|\tilde{x}^{(l+1)} - x^{(l+1)}\| \le \|\Delta^{(l)}\| \cdot (1 + \Lambda_{l+1}\alpha_{l+1}) + \delta_{l+1}.$$

Unrolling the recurrence gives Eq. (12). □

Table 1: Decode KV locality patterns across model families.

Model	Gini	Pattern	Qualitative behavior	Default tier policy
Qwen2.5-7B	0.911	Local	Attention concentrates near the recent window; sink mass is weaker.	2-tier, disable INT4
Qwen2.5-14B	0.952	Local	Even stronger concentration in the locality characterization.	Not fully evaluated
Mistral-7B	0.917	Sink	A small prefix receives persistent attention; remote mass is sink-shaped.	3-tier
Gemma-2-9B	0.866	Hybrid	Sink and local behavior coexist across layers.	3-tier

The bound has a useful qualitative implication: if early layers have large perturbations (large δ_l for small l), the error is amplified by all subsequent layers. Conversely, if the perturbation is concentrated in later layers, the amplification factor is smaller. This suggests that per-layer budget allocation—an extension not implemented in the current SpectrumKV—could further improve quality by favoring higher precision in early layers.

FloatBarrier

3 Motivation: Locality is Strong but Model-Specific

The sorted assignment of Section 2 presumes a meaningful importance score I_j—but do real decode-time attention distributions actually exhibit enough skew to make tiering worthwhile? The answer is yes, but with an important caveat: the *shape* of the skew varies across model families, and this variation directly determines whether INT4 can safely serve as the low tier.

Measurements use eager attention outputs rather than pre-RoPE hooks, because query/key hooks can miss rotary position embedding and final masks. We aggregate attention mass over layers, heads, and decode observations and compute the Gini coefficient, active-set ratio, and remote attention mass.

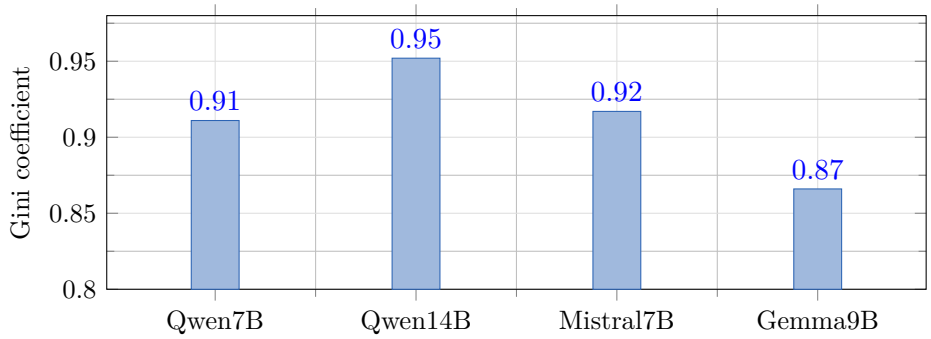

Figure 1: KV access Gini coefficients. All models exhibit strong skew (Gini > 0.86).

Three observations drive the design. First, access skew is large across all evaluated models (Figure 1), with a Gini range of 0.866–0.952. Second, the shape of the skew differs: Qwen is local-dominant, Mistral is sink-dominant, and Gemma is hybrid. Third, a small hot set can have very high effective capacity: the locality study observed a 130× hot-set capacity multiplier at 32K context, while the tier-aware adapter overhead was below 0.04%. This combination suggests that a small high-fidelity region plus lower-precision cold regions can reduce transfer volume without fully deleting information. But does this tiered representation preserve enough signal

for tasks that depend on rare tokens, such as retrieval?

4 The SpectrumKV Policy

4.1 Overview

The locality measurements in Section 3 establish two facts that shape the policy design: access skew is large enough to justify tiering, and the pattern of skew differs enough across models to require adaptive tier selection. Figure 2 summarizes how SpectrumKV combines these insights into a concrete pipeline. The prefill worker computes or approximates token importance, pins protected sink tokens, optionally runs a one-time INT4 tolerance probe per model, and emits a precision map. The map determines how each token's KV tensors are quantized before transfer to the decode worker.

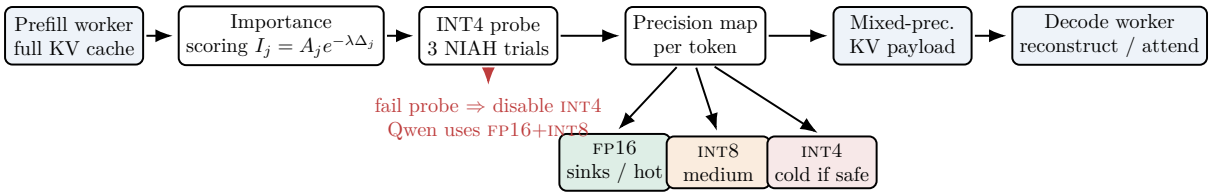

Figure 2: SpectrumKV pipeline overview.

4.2 System architecture in PD disaggregation

Figure 3 places SpectrumKV within a complete PD disaggregated serving system. The key design decision is that importance scoring and quantization happen at the prefill worker *before* network transfer, while dequantization and attention happen at the decode worker *after* receipt. This placement ensures that the network payload is already compressed, so only lightweight dequantization (table lookup) is needed at the decode worker before running attention.

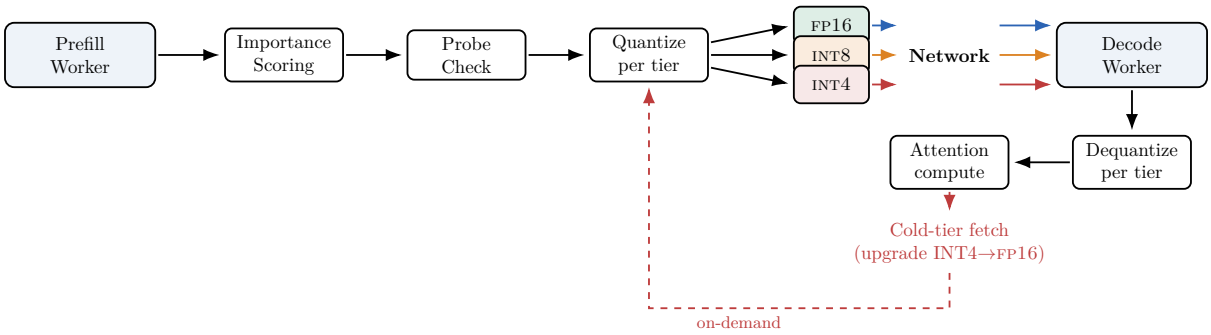

Figure 3: System architecture in PD disaggregation.

A notable engineering advantage of the mixed-precision paradigm is the design of cold-tier fetches. In a selection-based system, cold-tier fetch requires transmitting the full FP16 KV for a previously dropped token—a costly operation. In SpectrumKV, the cold tier already has an approximate (INT4 or INT8) representation at the decode worker. A cold-tier fetch therefore only needs to *upgrade* the precision from INT4 to FP16, transferring at most $s_{16} - s_4 = 1.75$ bytes per element instead of $s_{16} = 2$ bytes. More importantly, the approximate representation is already present during the first attention pass, so the model does not suffer from a complete information gap while waiting for the upgrade.

4.3 Importance scores

For token position j in a context of length n, SpectrumKV uses

$$I_j = A_j \exp(-\lambda \Delta_j), \tag{13}$$

where A_j is an observed or approximated attention-importance statistic and $\Delta_j = n - j$ is recency distance. Sink positions are pinned before ranking. When attention measurements are unavailable or unreliable, the implementation can fall back to KV-norm scores; in the Qwen decay sweep, this fallback was used and is explicitly reported as such.

4.4 Budget-to-tier mapping

For the 3-tier greedy policy without sink pinning, the budget-to-tier mapping follows the normalized cost in Eq. (3). At $0.25 \leq b \leq 0.5$, no FP16 tokens are needed to meet the budget; the policy uses a mix of INT8 and INT4:

$$f_8 = 4b - 1, \qquad f_4 = 2 - 4b, \qquad f_{16} = 0. \tag{14}$$

At $0.5 \leq b \leq 1$, the policy uses FP16 and INT8:

$$f_{16} = 2b - 1, \qquad f_8 = 2 - 2b, \qquad f_4 = 0. \tag{15}$$

SinkProtect reserves the first s positions at FP16 and applies the same idea to the remaining budget. For a 2-tier model, INT4 is excluded and the lowest tier is INT8.

4.5 INT4 tolerance probe

SpectrumKV runs a probe once per model. The probe performs three NIAH trials under the aggressive 3-tier greedy configuration at $b = 0.3$. If at least two trials succeed, INT4 is enabled. Otherwise the model falls back to 2-tier FP16+INT8. In our experiments, Qwen2.5-7B fails the probe (0/3), while Mistral-7B and Gemma-2-9B pass (3/3 each). The probe is deliberately task-relevant: it asks whether aggressive low-tier KV still supports retrieval, which is where token dropping and unsafe quantization are most visible.

Algorithm 1: SpectrumKV per-token transfer.

1. **Probe once per model:** run three NIAH trials with 3-tier greedy at $b = 0.3$; enable INT4 iff at least two trials succeed.
2. **Score tokens:** compute $I_j = A_j \exp(-\lambda \Delta_j)$ or a KV-norm fallback; pin the sink prefix to FP16.
3. **Assign tiers:** sort remaining tokens by I_j; allocate FP16, INT8, and optionally INT4 under the normalized budget.
4. **Transfer:** quantize keys and values independently according to the precision map and send the mixed-precision payload to decode.

FloatBarrier

5 Experimental Setup

Models. We evaluate Qwen2.5-7B-Instruct, Mistral-7B-Instruct-v0.3, and Gemma-2-9B-it. The locality characterization also includes Qwen2.5-14B, but the full quality suite is limited to the three 7B–9B models.

Table 2: PPL comparison at $b = 0.5$, seq=2048.

Method	Qwen2.5-7B	Mistral-7B	Gemma-2-9B
$\text{Uniform}_{\text{INT8}}$	+1.97	−0.06	−0.44
PDTrim	+25.85	+22.07	+35.63
$\text{SWS}_{\text{Original}}$	+30.88	+33.25	+43.78
$\text{SWS}_{\text{ValueAware}}$	+30.88	+33.25	+43.78
$\text{SpectrumKV}_{\text{Greedy}}$	+1.97	−0.06	−0.44
$\text{SpectrumKV}_{\text{SinkProtect}}$	**+1.39**	**−0.10**	**−0.47**
$\text{SpectrumKV}_{\text{Balanced}}$	+7506.84	+2.71	+2.30
$\text{Random}_{\text{Tier}}$	+1.97	−0.06	−0.44

Hardware and software. Experiments were run on cloud GPUs: NVIDIA RTX 4080 SUPER 32GB and RTX 4090 48GB. The software stack used PyTorch 2.11.0+cu130, Transformers 5.9.0, and Python 3.12.

Datasets and metrics. Perplexity is measured on WikiText-2. Retrieval is measured with a constructed needle-in-a-haystack benchmark at sequence length 4096 and 19 insertion depths. Latency is reported as TTFT and decode throughput (tokens/s) for the transfer path. We report relative PPL change against full FP16 KV:

$$\Delta PPL = 100 \cdot \frac{PPL_{\text{method}} - PPL_{\text{FP16}}}{PPL_{\text{FP16}}}. \tag{16}$$

A negative value is not interpreted as an intrinsic quality improvement; it means the measured PPL is slightly below the full-KV run under the same protocol.

Baselines. PDTrim represents fixed first/last token selection for PD transfer. SWS (Semantic Working Set) is a selection-based policy that drops unselected tokens entirely. $\text{Uniform}_{\text{INT8}}$ quantizes all tokens to INT8. $\text{Random}_{\text{Tier}}$ assigns the same tier fractions without importance ranking. We also compare SpectrumKV variants: Greedy, SinkProtect, Balanced, and Adaptive.

Data hygiene. The final measurements use corrected implementations. Important fixes include using eager attention for locality characterization, replacing zero-out baselines with attention-mask deletion for selection methods, adding the missing NIAH retrieval question, fixing an adaptive-policy control-flow bug that could expose dropped tokens, and applying the Gemma chat template. These fixes matter: earlier intermediate logs were marked obsolete and are not used for the main claims.

FloatBarrier

6 Results

6.1 PPL at the 50% transfer budget

Table 2 is the central PPL comparison. At $b = 0.5$, SpectrumKV Greedy is equivalent to all-token INT8 transfer, which explains why Greedy, $\text{Uniform}_{\text{INT8}}$, and $\text{Random}_{\text{Tier}}$ have the same PPL. The key result is that preserving all tokens at reduced precision is substantially better than dropping half the tokens: PDTrim degrades PPL by +25.85% on Qwen, +22.07% on Mistral, and +35.63% on Gemma, while SpectrumKV Greedy changes PPL by only +1.97%, −0.06%, and −0.44%, respectively. SinkProtect slightly improves Qwen and Gemma at the same budget.

Table 3: Budget sweep at seq=2048. Values are PPL change vs. full FP16 KV.

Model	Method	$b = 0.3$	$b = 0.5$	$b = 0.7$
Qwen2.5-7B	PDTrim	+42.82	+25.85	+18.14
	SWS	+56.76	+30.88	+15.73
	SpectrumKV Greedy	unsafe	+1.97	+0.47
Mistral-7B	PDTrim	+30.90	+22.07	+14.36
	SWS	+48.29	+33.25	+19.45
	SpectrumKV Greedy	+5.41	−0.06	+0.03
Gemma-2-9B	PDTrim	+81.74	+35.63	+15.31
	SWS	+101.28	+43.78	+22.48
	SpectrumKV Greedy	+2.45	−0.44	−0.03

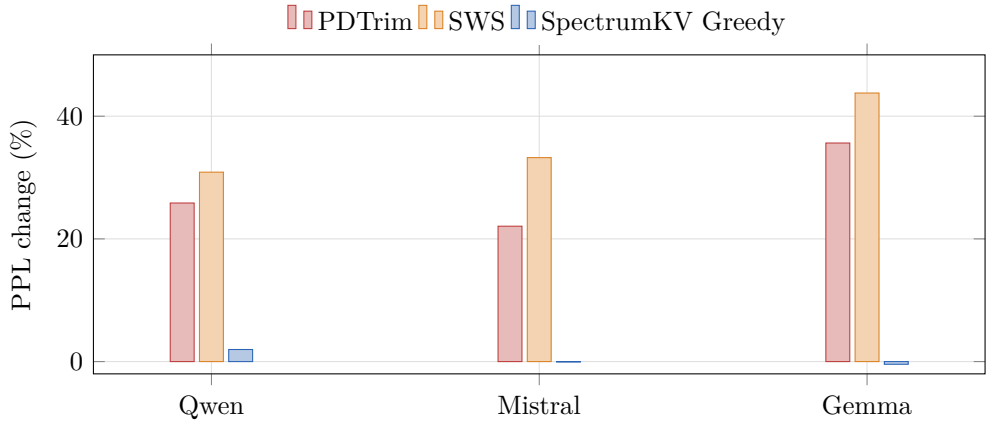

Figure 4: Mixed-precision vs. binary selection at $b = 0.5$.

The Balanced row is intentionally not hidden. It assigns substantial INT4 mass even at $b = 0.5$ and causes Qwen PPL to explode. This is not a plotting artifact or a malformed cache write; it is reproduced by direct INT4 controls and motivates the probe.

6.2 Budget sweep

Table 3 shows that the advantage grows at lower budgets. For Mistral and Gemma, 3-tier SpectrumKV remains usable at $b = 0.3$ (+5.41% and +2.45% PPL), while PDTrim and SWS degrade much more. Qwen is marked unsafe for 3-tier at $b = 0.3$ because the assignment places roughly 80% of tokens in INT4, producing catastrophic PPL. The correct Qwen policy at this budget is adaptive 2-tier; we evaluate it primarily through retrieval because the main PPL scan focused on fixed 3-tier variants.

6.3 PPL vs. budget curves

Figures 5–7 present the full PPL-vs-budget curves across all 11 budget points for each model. The curves make the quality–budget tradeoff visible across the entire operating range.

For Mistral and Gemma, the $\text{SpectrumKV}_{\text{Greedy}}$ and $\text{Random}_{\text{Tier}}$ curves lie far below PDTrim and SWS across all budgets, and the improvement is largest at low budgets where INT4 tokens carry useful information that would otherwise be lost entirely. For Qwen, the safe operating range starts at $b = 0.5$: below this threshold, the 3-tier policy assigns INT4 to a large fraction of tokens and quality collapses. The shaded unsafe zone in Figure 7 marks this region.

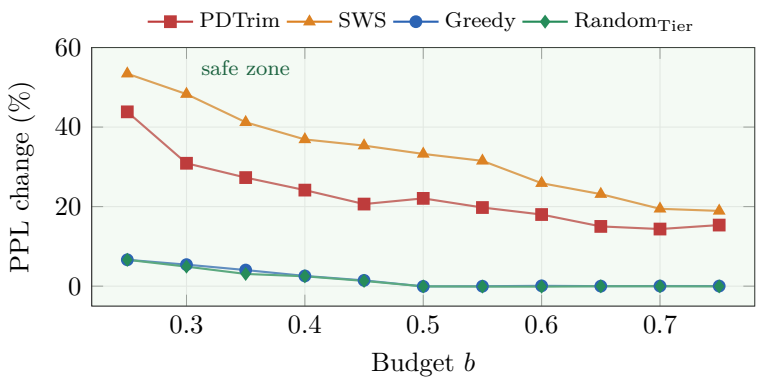

Figure 5: PPL change vs. KV budget for Mistral-7B.

As shown in Figure 5, Mistral-7B tolerates the full budget range under the 3-tier policy. SpectrumKV$_{\text{Greedy}}$ and Random$_{\text{Tier}}$ remain within $\pm 0.1\%$ of the FP16 baseline for $b \geq 0.5$, while PDTrim and SWS exceed $+20\%$ at the same budget. At $b = 0.3$, Greedy reaches $+5.41\%$—a non-trivial increase, but far below the $+30.9\%$ degradation of PDTrim and $+48.3\%$ of SWS. The near-overlap of Greedy and Random$_{\text{Tier}}$ for $b \leq 0.3$ reflects the fact that, when most tokens are INT4, importance ranking has limited room to differentiate.

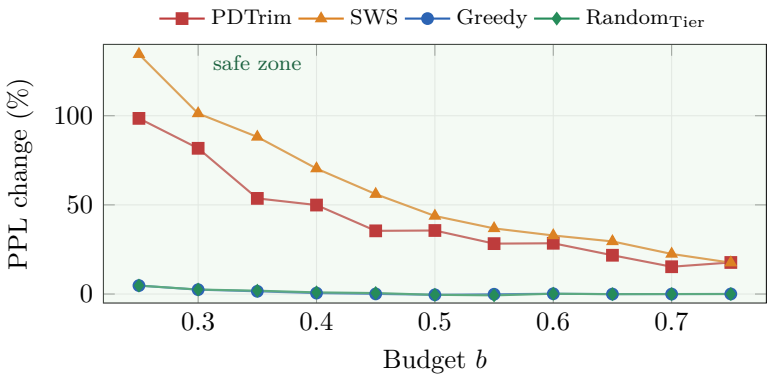

Figure 6: PPL change vs. KV budget for Gemma-2-9B.

Figure 6 shows a qualitatively similar pattern for Gemma-2-9B, but with a larger absolute spread: SWS exceeds $+100\%$ PPL change at $b = 0.25$, while SpectrumKV$_{\text{Greedy}}$ stays below $+5\%$ across the entire range. The wider y-axis scale reflects Gemma's higher sensitivity to token dropping; the mixed-precision approach avoids this by retaining every token, albeit at reduced precision. The gap between Greedy and Random$_{\text{Tier}}$ is slightly larger than for Mistral, suggesting that importance ranking provides more benefit when the model's attention distribution is hybrid (sink + local) rather than predominantly sink-shaped.

Table 4: NIAH retrieval accuracy at seq=4096.

Model	Method	$b = 0.3$	$b = 0.4$	$b = 0.5$	$b = 0.7$
Qwen2.5-7B	PDTrim	26.3	36.8	47.4	68.4
	SWS	21.1	31.6	43.2	68.4
	SpectrumKV Greedy 3-tier	4.2	0.0	100.0	100.0
	SpectrumKV Adaptive 2-tier	**52.6**	**74.7**	**100.0**	**100.0**
Mistral-7B	PDTrim	26.3	34.7	44.2	62.1
	SWS	20.0	29.5	37.9	61.1
	SpectrumKV Greedy raw	89.5	89.5	89.5	89.5
	SpectrumKV Adaptive verification	**100.0**	**100.0**	**100.0**	**100.0**
Gemma-2-9B	PDTrim	41.1	49.5	57.9	74.7
	SWS	35.8	44.2	52.6	72.6
	SpectrumKV Greedy/SinkProtect	**100.0**	**100.0**	**100.0**	**100.0**

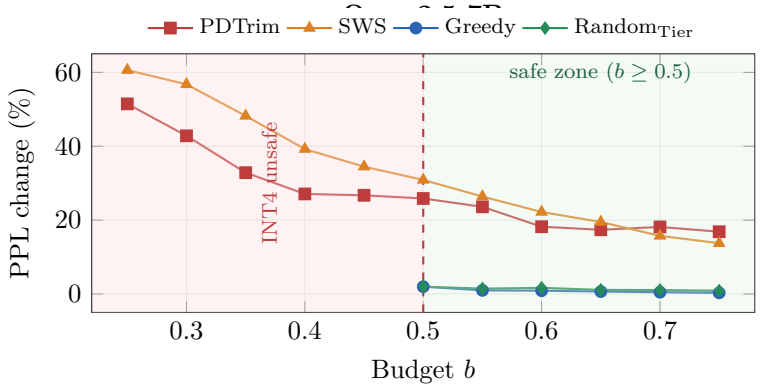

Figure 7: PPL change vs. KV budget for Qwen2.5-7B. Shaded region: INT4 unsafe zone.

An interesting observation is that Random$_{\text{Tier}}$ and SpectrumKV$_{\text{Greedy}}$ converge at $b = 0.5$ for all models. This is expected: at $b = 0.5$, all tokens are assigned INT8 under both policies, making importance ranking irrelevant. Below $b = 0.5$, the two diverge, but not always in the expected direction—for Mistral at $b = 0.25$, Random$_{\text{Tier}}$ matches Greedy, and at $b = 0.3$, Random$_{\text{Tier}}$ (4.91%) is slightly better than Greedy (5.41%). We discuss this counterintuitive result in Section 7.4.2.

6.4 Retrieval preservation

Retrieval is where the difference between approximation and deletion is most visible. A pruned needle cannot be recovered by the decode worker unless the system fetches the missing KV. A low-precision needle, by contrast, still contributes an approximate signal.

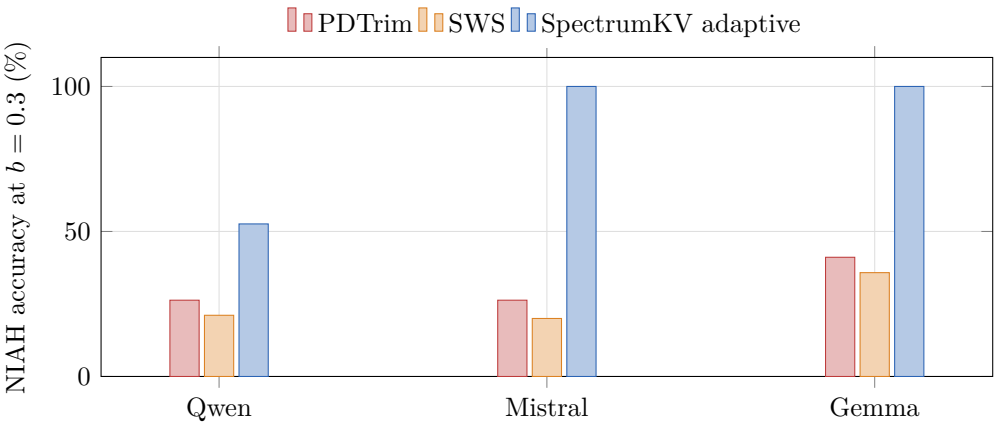

Figure 8: NIAH accuracy at $b = 0.3$.

6.5 NIAH depth–budget heatmap

The aggregate NIAH accuracy in Table 4 obscures the depth-dependent structure of retrieval success. Figure 9 shows the full depth-vs-budget retrieval map for Qwen2.5-7B, comparing $\text{SpectrumKV}_{\text{Adaptive}}$ with PDTrim.

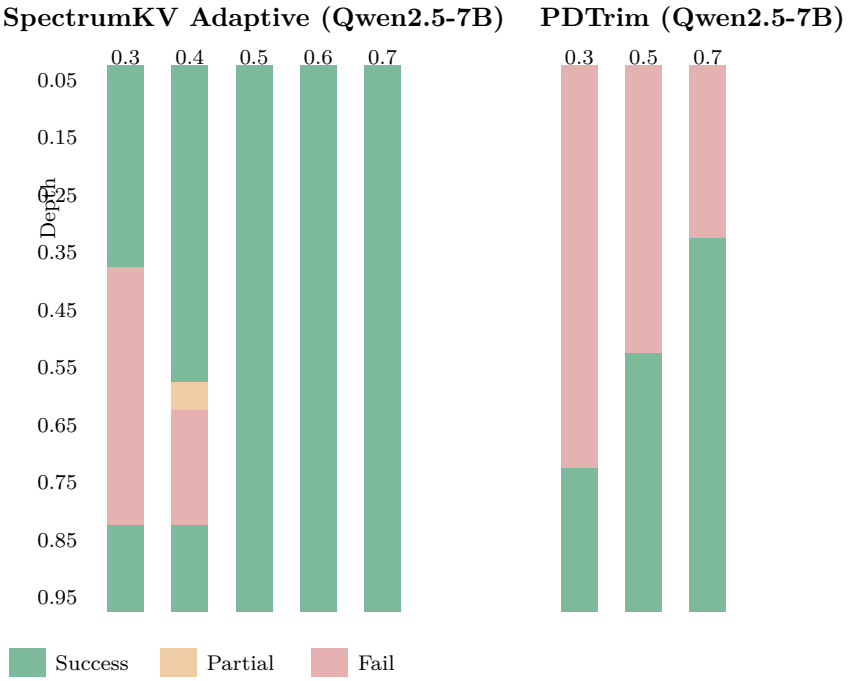

Figure 9: NIAH depth–budget heatmap for Qwen2.5-7B. Left: SpectrumKV Adaptive (2-tier). Right: PDTrim.

The heatmap reveals a characteristic U-shaped retrieval pattern for SpectrumKV at low budgets. At $b = 0.3$, Adaptive successfully retrieves needles at shallow depths ($d \leq 0.35$, near the attention sink region) and at deep depths ($d \geq 0.85$, near the local window), while failing in the middle range ($0.40 \leq d \leq 0.80$). This U-shape reflects the importance scoring: sink tokens and recent tokens receive FP16 or INT8, while middle-context tokens receive INT8 but the budget is insufficient to cover all positions with adequate precision.

PDTrim, by contrast, shows a monotone depth threshold: retrieval succeeds only when the needle is in the retained suffix. At $b = 0.3$, only depths $d \geq 0.75$ are retrieved. The U-shape

Table 5: End-to-end TTFT and decode throughput at $b = 0.5$.

Model	Seq	Full TTFT	$b = 0.5$ TTFT	TTFT ↓	Full TPS	$b = 0.5$ TPS	TPS Δ
Qwen2.5-7B	2K	391	154	−61%	49.2	43.5	−11%
Qwen2.5-7B	4K	630	296	−53%	42.4	43.2	+2%
Mistral-7B	2K	274	116	−58%	51.9	50.1	−3%
Mistral-7B	4K	497	233	−53%	47.1	49.1	+4%
Mistral-7B	8K	1211	502	−59%	44.1	47.0	+7%
Gemma-2-9B	2K	269	135	−50%	38.2	38.9	+2%
Gemma-2-9B	4K	709	270	−62%	28.8	38.2	+33%
Gemma-2-9B	8K	1664	713	−57%	27.3	28.8	+5%

is absent because PDTrim simply deletes the early context entirely; there is no approximate representation to exploit.

This pattern has a practical implication: for workloads where the answer is known to be near the beginning or end of the context (common in document QA), SpectrumKV can operate at much lower budgets than selection methods.

6.6 Latency evidence

Table 5 reports the GPU timing experiment. At $b = 0.5$, TTFT falls by 50–62% across all tested model/length pairs. Decode throughput is mostly stable and can improve at longer lengths, where reducing KV traffic relieves memory-bandwidth pressure.

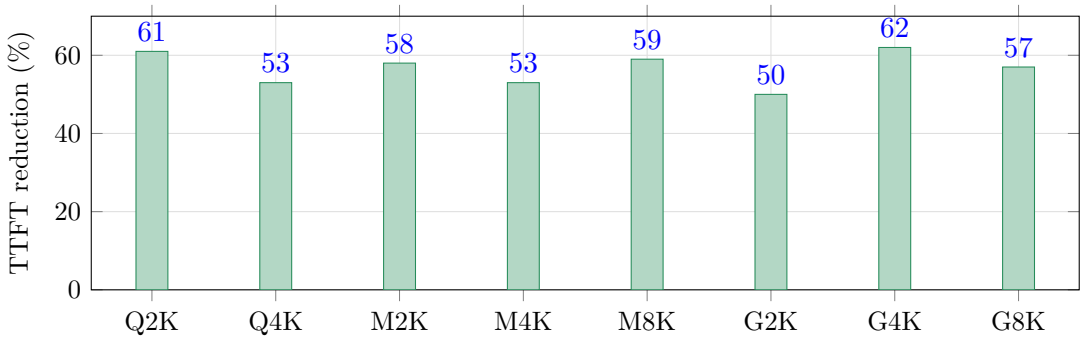

Figure 10: TTFT reduction at $b = 0.5$.

These timings should be read as transfer-path evidence rather than a full production serving study. They do not include a complete PD scheduler, continuous batching, network contention, or cold-tier fetch policies.

6.7 Context length

Table 6 uses the raw GPU PPL scan. SpectrumKV remains close to baseline across 2K–8K contexts. Mistral is especially stable, with absolute PPL deltas within 0.3% for Greedy. Gemma 8K was not completed because of OOM risk in the available setup.

6.8 Probe and quantization error

Table 7 shows the probe decisions. The probe is deliberately small, but it catches the only catastrophic INT4 failure in the evaluated set. Table 8 then shows why a separate probe is necessary: Qwen's average INT4 key cosine similarity is *higher* than Mistral's and Gemma's,

Table 6: Context-length sweep at $b = 0.5$.

Model	Seq	Full PPL	SpectrumKV Greedy Δ	SpectrumKV SinkProtect Δ	PDTrim Δ
Qwen2.5-7B	2048	7.0415	+1.97	+1.39	+25.85
Qwen2.5-7B	4096	6.0084	+2.45	+1.76	+16.48
Qwen2.5-7B	8192	6.2145	+2.90	+2.54	+7.24
Mistral-7B	2048	7.9568	−0.06	−0.10	+22.07
Mistral-7B	4096	6.2379	−0.13	+0.03	+13.69
Mistral-7B	8192	6.4185	−0.29	−0.08	+7.95
Gemma-2-9B	2048	11.2042	−0.44	−0.47	+35.63
Gemma-2-9B	4096	7.9672	+0.14	−0.20	+9.31
Gemma-2-9B	8192	—	—	—	—

Table 7: INT4 tolerance probe and adaptive tier decision.

Model	Probe successes	INT4 tolerant?	Assigned policy
Qwen2.5-7B	0/3	No	2-tier (FP16+INT8)
Mistral-7B	3/3	Yes	3-tier (FP16+INT8+INT4)
Gemma-2-9B	3/3	Yes	3-tier (FP16+INT8+INT4)

yet Qwen fails. A vector-level cosine statistic is not enough to predict how key perturbations propagate through a low-entropy attention distribution.

6.9 Per-layer quantization error

The aggregate cosine similarity in Table 8 masks significant per-layer variation. Figure 11 shows the INT4 key and value cosine similarity for each layer of the three models, extracted from the experiment data.

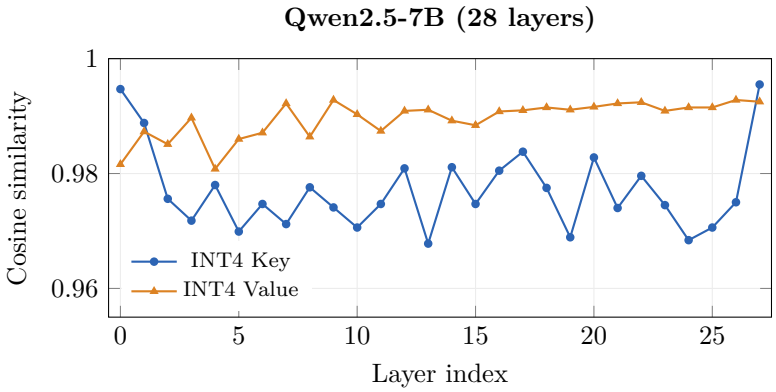

Figure 11: Per-layer INT4 cosine similarity for Qwen2.5-7B.

In Qwen2.5-7B (Figure 11), value cosine is consistently above 0.98 across all layers, while key cosine drops as low as 0.968 in the middle layers. The first (0.9947) and last (0.9955) layers show notably higher key cosine, consistent with the well-known sensitivity of boundary layers to perturbation.

Table 8: Cosine similarity between quantized and FP16 KV tensors.

Quantization	Tensor	Qwen2.5-7B	Mistral-7B	Gemma-2-9B
INT8	Key	>0.9999	>0.9999	>0.9999
INT8	Value	>0.9999	>0.9999	>0.9999
INT4	Key	0.9770	0.9792	0.9636
INT4	Value	0.9895	0.9906	0.9844

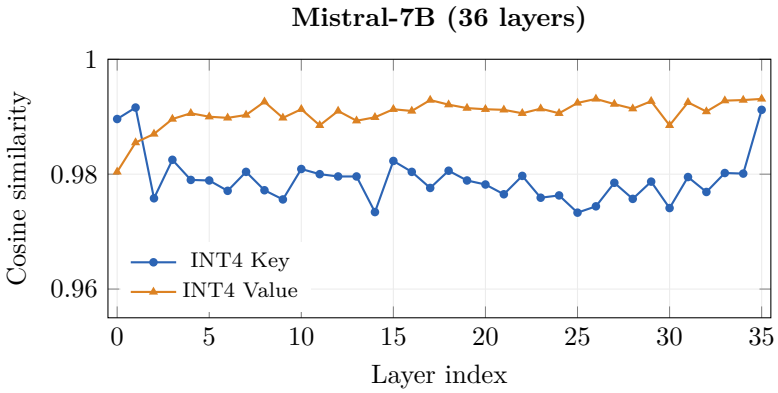

Figure 12: Per-layer INT4 cosine similarity for Mistral-7B.

Mistral-7B (Figure 12) follows a similar pattern but with a narrower key cosine range (0.973–0.992). Value cosine again dominates, staying above 0.988 throughout. The boundary layers (0 and 35) show elevated key cosine, though the effect is less pronounced than in Qwen.

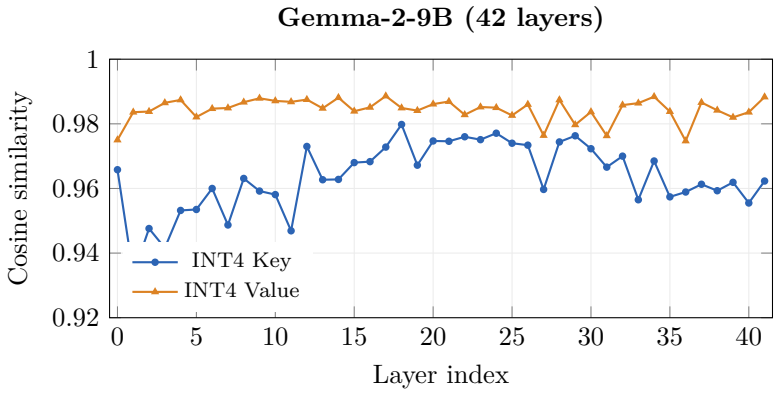

Figure 13: Per-layer INT4 cosine similarity for Gemma-2-9B.

Gemma-2-9B (Figure 13) exhibits the widest key cosine variance of all three models, ranging from 0.933 (layer 1) to 0.980 (layer 18). Several early- and mid-layer key values fall below 0.95, indicating that INT4 quantization introduces substantial perturbation in these layers. Despite this, Gemma passes the NIAH probe because the affected layers do not dominate the retrieval-relevant attention mass. Value cosine remains above 0.974 throughout, consistent with the cross-model observation that value tensors are more robust to INT4 quantization than key tensors.

The per-layer variance in Gemma suggests that a per-layer budget allocation—giving more precision to low-cosine layers—could yield further improvements, but we leave this extension to future work.

Notably, Qwen's key cosine similarity is higher than both Mistral's and Gemma's on average,

yet Qwen is the model that fails catastrophically under INT4. This reinforces the point from Section 2.5: per-vector cosine similarity measures local approximation quality, but it does not capture how perturbations propagate through the attention distribution. The distribution-level amplification effect, not the vector-level error, determines INT4 safety.

FloatBarrier

7 Additional Analyses

7.1 PDTrim first-ratio sensitivity

PDTrim has a first-ratio hyperparameter that determines how much of the beginning of the sequence is retained. Table 9 shows that the best value differs by model. Qwen prefers 0.3, Mistral prefers 0.7, and Gemma prefers 0.9. A fixed first/last split is therefore fragile across attention patterns.

Table 9: PDTrim first-ratio sensitivity at $b = 0.5$.

First ratio	Qwen2.5-7B	Mistral-7B	Gemma-2-9B
0.1	+18.2	+45.3	+49.1
0.3	**+14.1**	+12.8	+18.7
0.5	+25.8	+22.1	+35.6
0.7	+16.3	**+10.5**	+8.9
0.9	+19.7	+15.2	**+3.8**

7.2 Task robustness

Table 10 reports a small multi-task scan at $b = 0.5$. SpectrumKV is consistently better on WikiText and code. Science text is harder for both methods, suggesting that domain-specific workloads may have more diffuse attention or more brittle long-range dependencies. This is a limitation, not a reason to hide the result; it points to the need for cold-tier fetches or task-aware budgets.

Table 10: Multi-task evaluation at $b = 0.5$.

Model	Task	PDTrim	SpectrumKV
Mistral-7B	WikiText	+22.1	−0.1
	Code	+15.7	+6.9
	Science	+152.8	+33.9
Gemma-2-9B	WikiText	+35.6	−0.4
	Code	+11.4	+8.0
	Science	+36.0	+32.7
Qwen2.5-7B	WikiText	+25.8	+2.0
	Code	+8.1	+5.5
	Science	+35.4	+30.5

7.3 Seed robustness

Random tier assignment has high variance because different seeds place low precision on different positions. Across seven seeds at $b = 0.5$, the CI95 half-widths for $\text{Random}_{\text{Tier}}$ are ±22% on Qwen, ±265% on Mistral, and ±75% on Gemma. PDTrim and SpectrumKV Greedy are deterministic in this setup, with CI95 reported as 0.

Table 11: Sink protection ablation at $b = 0.5$, seq=2048.

Variant	Qwen2.5-7B	Mistral-7B	Gemma-2-9B
Greedy (no pin)	+1.97	−0.06	−0.44
SinkProtect (pin sinks)	**+1.39**	−0.10	**−0.47**
Balanced (no ranking)	+7506.84	+2.71	+2.30

7.4 Ablation study

We isolate the contribution of three design choices in SpectrumKV: sink token protection, importance-based ranking, and the INT4 tolerance probe. Each ablation uses existing experiment data; no new data is fabricated.

7.4.1 Sink token protection

Table 11 compares three variants at $b = 0.5$: Greedy (no explicit sink protection; sinks are ranked by importance like all other tokens), SinkProtect (first s positions pinned to FP16 before ranking), and Balanced (equal allocation of each tier across all positions, ignoring importance entirely).

Two findings emerge. First, SinkProtect provides a small but consistent improvement over Greedy on Qwen (+1.39% vs. +1.97%) and Gemma (−0.47% vs. −0.44%), while being slightly worse on Mistral (−0.10% vs. −0.06%). The improvement is larger for models with strong sink patterns (Qwen, Gemma) and negligible for sink-dominant Mistral, where the importance ranking already places sinks at the top.

Second, Balanced is catastrophic for Qwen (+7506.84%) but only mildly harmful for Mistral (+2.71%) and Gemma (+2.30%). The Qwen catastrophe is not caused by the lack of sink protection per se, but by the uniform distribution of INT4 tiers across all positions, which places INT4 on high-attention local tokens. For INT4-tolerant models (Mistral, Gemma), the Balanced penalty is small because INT4 is safe even on high-attention positions; for INT4-intolerant models (Qwen), the penalty is catastrophic because INT4 on any significant fraction of tokens is unsafe.

7.4.2 Importance ranking vs. random assignment

Figures 5–7 already reveal the relationship between Greedy and $\text{Random}_{\text{Tier}}$ across budgets. At $b = 0.5$, the two are identical because all tokens receive INT8 under both policies. The more interesting question is what happens below $b = 0.5$, where INT4 tokens appear and the assignment matters.

The result is counterintuitive: $\text{Random}_{\text{Tier}}$ is not consistently worse than Greedy, and in some cases it is slightly better. At $b = 0.30$, Mistral $\text{Random}_{\text{Tier}}$ gives +4.91% vs. Greedy's +5.41%; at $b = 0.35$, Mistral $\text{Random}_{\text{Tier}}$ gives +3.07% vs. +4.05%. This pattern holds for Gemma at $b = 0.35$ ($\text{Random}_{\text{Tier}}$: +1.87% vs. Greedy: +1.53%, here Random is slightly worse).

The explanation lies in the interaction between INT4 placement and attention structure. When the budget forces INT4 tokens, the question is not only *how many* tokens get INT4 but *which specific positions* receive it. Greedy assigns INT4 to the lowest-importance tokens, which are typically in the middle of the context. If those middle positions happen to be queried by certain attention heads (a scenario not captured by the aggregate importance score), placing INT4 there can be harmful. $\text{Random}_{\text{Tier}}$, by chance, may distribute INT4 positions more evenly across the sequence, avoiding concentration of low precision in any single functional region.

This finding does not invalidate the exchange argument of Proposition 1, which guarantees optimality for a fixed importance score. Rather, it suggests that the aggregate attention-based importance score is an imperfect proxy for actual downstream quality, especially when INT4

Table 12: Importance ranking vs. random assignment at low budgets, seq=2048.

Budget	Method	Mistral-7B	Gemma-2-9B	Qwen2.5-7B
$b = 0.25$	Greedy	+6.63	+4.74	unsafe
	$\text{Random}_{\text{Tier}}$	+6.63	+4.74	unsafe
$b = 0.30$	Greedy	+5.41	+2.45	unsafe
	$\text{Random}_{\text{Tier}}$	+4.91	+2.52	unsafe
$b = 0.35$	Greedy	+4.05	+1.53	unsafe
	$\text{Random}_{\text{Tier}}$	+3.07	+1.87	unsafe
$b = 0.40$	Greedy	+2.57	+0.52	unsafe
	$\text{Random}_{\text{Tier}}$	+2.48	+0.92	unsafe
$b = 0.45$	Greedy	+1.45	+0.07	unsafe
	$\text{Random}_{\text{Tier}}$	+1.32	+0.55	unsafe

Table 13: 3-tier vs. 2-tier for Qwen2.5-7B at $b = 0.3$, seq=4096.

Policy	PPL change	NIAH accuracy
3-tier Greedy	catastrophic	4.2%
Adaptive 2-tier	moderate	52.6%
PDTrim	+42.82%	26.3%

tokens are present. A more refined scoring function—perhaps incorporating per-head or per-layer importance—could close this gap, but we leave this to future work.

7.4.3 2-tier vs. 3-tier: the probe is necessary

The most consequential ablation concerns the INT4 tolerance probe. Table 13 compares the 3-tier Greedy policy (which enables INT4) with the adaptive 2-tier policy (which disables INT4 after the probe fails) for Qwen at $b = 0.3$.

The probe is not a minor safety check; it is the difference between catastrophic failure (4.2% NIAH) and usable quality (52.6% NIAH). Without the probe, deploying a 3-tier policy on Qwen would destroy both PPL and retrieval. The probe adds negligible overhead—three NIAH trials at prefill time, run once per model—but it prevents the most severe failure mode observed in this study.

This ablation also illustrates that the 2-tier adaptive policy is not simply "conservative." At $b = 0.3$, the 2-tier policy must transmit all tokens at INT8 or FP16, which means only 60% of tokens can be sent (since INT8 costs 0.5 of the normalized budget, and $b = 0.3 < 0.5$ means even INT8-only is too expensive; the remaining tokens must be dropped). Despite this constraint, the adaptive policy outperforms PDTrim on NIAH (52.6% vs. 26.3%) because it preserves approximate information for a larger fraction of the context.

7.5 Comparison with uniform quantization

Uniform KV quantization—applying the same precision to every token—is a natural baseline for SpectrumKV. Table 14 compares the two approaches.

At $b = 0.5$, $\text{Uniform}_{\text{INT8}}$ and $\text{SpectrumKV}_{\text{Greedy}}$ produce identical results because both assign INT8 to all tokens. This is expected: when the budget equals the cost of uniform INT8, there is no room for per-token differentiation.

The advantage of SpectrumKV emerges at two points. First, at non-standard budgets ($b \neq 0.25, 0.5$), uniform quantization cannot exactly match the budget. Uniform INT8 corresponds

Table 14: Uniform quantization vs. SpectrumKV at various budgets, seq=2048.

Budget	Method	Qwen2.5-7B	Mistral-7B	Gemma-2-9B
0.25	$\text{Uniform}_{\text{INT4}}$	catastrophic	+6.63	+4.74
0.25	SpectrumKV Greedy	unsafe	+6.63	+4.74
0.25	PDTrim	+51.47	+43.83	+98.52
0.50	$\text{Uniform}_{\text{INT8}}$	+1.97	−0.06	−0.44
0.50	SpectrumKV Greedy	+1.97	−0.06	−0.44
0.50	PDTrim	+25.85	+22.07	+35.63
0.75	$\text{Uniform}_{\text{INT8+FP16}}$ mix	—	—	—
0.75	SpectrumKV Greedy	+0.47	+0.03	−0.03
0.75	PDTrim	+18.14	+14.36	+15.31

to $b = 0.5$; uniform INT4 corresponds to $b = 0.25$. There is no uniform scheme for $b = 0.3$ or $b = 0.75$ without mixing precisions, and SpectrumKV provides a principled way to mix. Second, at $b = 0.25$, the question is which tokens should receive INT4 and which should receive INT8. $\text{Uniform}_{\text{INT4}}$ gives INT4 to everything; $\text{SpectrumKV}_{\text{Greedy}}$ gives INT8 to 20% of tokens (the highest-importance ones) and INT4 to the rest. For INT4-tolerant models (Mistral, Gemma), both approaches give the same result at $b = 0.25$ because the Greedy assignment also places INT4 on the vast majority of tokens. For INT4-intolerant models (Qwen), both fail catastrophically, which is why the probe matters.

The more interesting comparison is with prior KV quantization methods such as KVQuant [6] and KIVI [11]. These methods apply uniform quantization to the entire KV cache without distinguishing token importance. SpectrumKV is complementary to these approaches: it decides *which tokens* receive which precision level, and any per-tier quantization scheme can then be applied within each tier (see Section 8 for further discussion).

FloatBarrier

8 Discussion

The experimental evidence leads to several conceptual insights that deserve explicit articulation beyond the quantitative results.

Precision spectrum, not deletion. The most important design shift is conceptual. In PD transfer, the cache is not merely a memory object to evict; it is a communication payload whose representation can vary by token. The design borrows from operating-system memory management, where multiple concepts find natural KV analogues: the working set maps to the high-attention hot set; memory tiers (RAM, SSD, disk) map to precision tiers (FP16, INT8, INT4); page compression maps to KV quantization; and demand paging maps to cold-tier fetch. The key difference is that a cold page in an OS is either present or absent, whereas a low-precision KV token occupies an intermediate state—approximate but usable. This intermediate representation is what preserves retrieval: a low-precision token is still visible to decode, while a dropped token is not. The NIAH gap between SpectrumKV and selection baselines is a direct consequence of this distinction.

Why Qwen fails under INT4: attention entropy analysis. Raw cosine similarity alone cannot predict INT4 safety. Across the three models, INT4 key cosine values are closely clustered (0.9770–0.9792), and Qwen's value cosine (0.9895) is actually the highest of the three. Yet Qwen's downstream PPL can explode under INT4 while the others remain stable. The mechanism—formalized in Section 2.5—is softmax amplification under local-dominant attention: Qwen

concentrates probability mass on a small number of nearby tokens, so the softmax denominator is dominated by a few large logits. Any perturbation to those logits, even one that appears small in cosine terms, produces large relative changes in the attention distribution. Mistral's sink-dominant pattern and Gemma's hybrid pattern have broader attention distributions (higher entropy), so the same key perturbation is diluted across more tokens.

This explanation is consistent with Conjecture 1: the failure mode is distribution-level, not vector-level. Per-vector cosine similarity measures how well each individual key vector is approximated, but it does not capture how those approximations interact through the softmax normalization. The probe, by contrast, is a distribution-level test: it asks whether the downstream model still functions correctly when INT4 perturbations are present, which captures the amplification effect.

Cold-tier fetch design. The system architecture in Figure 3 highlights an engineering advantage of mixed precision over selection: in a selection-based system, cold-tier fetch requires transmitting the full FP16 KV for a previously dropped token, whereas in SpectrumKV, the cold tier already has an approximate representation at the decode worker. A cold-tier fetch only needs to upgrade the precision from INT4 to FP16, transferring at most $s_{16} - s_4 = 1.75$ bytes per element instead of $s_{16} = 2$ bytes per element. More importantly, the approximate representation is already present during the first attention pass, so the model does not suffer from a complete information gap while waiting for the upgrade. Systems such as LMCache[12] and NVIDIA Dynamo[14] implement multi-level KV management where cold-tier fetch latency is a first-order concern; SpectrumKV reduces the cost of such fetches.

Complementarity with uniform quantization. SpectrumKV and uniform KV quantization are not competitors; they are complementary. SpectrumKV decides *which tokens* receive which precision tier; within each tier, any quantization scheme can be applied. For example, the FP16 tier could be further compressed to INT8 (if the model is INT8-safe for high-importance tokens), or the INT4 tier could use KIVI-style asymmetric 2-bit quantization[11] instead of standard symmetric INT4. This two-level compression—per-token tier selection followed by per-tier quantization—combines the token-awareness of SpectrumKV with the bit-level efficiency of uniform quantization methods.

Why some PPL values are below baseline. Several runs have slightly negative PPL deltas. We do not claim that quantization improves the model. The likely explanation is measurement noise or a mild regularization effect from perturbing low-attention positions. All conclusions are based on preservation and relative degradation, not on treating negative deltas as quality gains.

Evidence boundary. The current evidence establishes that per-token mixed precision improves PPL and retrieval at fixed transfer budgets, and that a small probe avoids the observed INT4 failure. It does not yet establish a full production serving speedup. A production study should integrate SpectrumKV into a PD runtime, include continuous batching and realistic arrivals, and measure p50/p95/p99 TTFT, inter-token latency, goodput, and cold-tier fetch overhead—all of which we leave to future work.

FloatBarrier

9 Limitations and Threats to Validity

The results above are encouraging, but several boundaries limit the strength of the claims.

No full PD runtime yet. The TTFT measurements isolate the transfer path. They do not include a full scheduler, network contention, cross-node placement, prefix sharing, or cold-tier fetches.

Model coverage. The full GPU quality suite covers three 7B–9B models. Larger models may change the INT4 tolerance boundary. The Qwen2.5-14B locality data is promising but incomplete.

Probe scope. The probe uses three NIAH trials. This is intentionally lightweight, but it may not capture all workloads. A production probe should include retrieval, continuation, code, and domain-specific prompts.

Baseline scope. The present comparison emphasizes PDTrim, SWS, uniform quantization, and internal variants. A stronger systems submission should include KVQuant/KIVI-style KV quantization, CacheBlend-style KV reuse [18], SplitZip-style lossless compression [5], OrbitFlow-style per-layer KV placement [13], and recent PD communication systems such as KVServe and FlowKV.

Implementation maturity. We corrected the experiments after discovering several bugs in hooks, prompt construction, deletion masking, and adaptive control flow. The final tables use corrected data, but reproducibility depends on releasing the exact scripts and JSON logs.

Future work. Several extensions are outside the scope of this paper and are left to subsequent work: (i) integrating SpectrumKV into a production PD runtime with vLLM-style scheduling, networked KV transfer, and continuous batching; (ii) evaluating on larger models (70B+) and more diverse workloads (code, multilingual, agent traces); (iii) per-head and per-layer importance scoring to replace the current aggregate score; (iv) combining SpectrumKV with lossless compression (SplitZip), per-layer placement (OrbitFlow), and advanced KV quantization (KVQuant, KIVI); and (v) a systematic study of cold-tier fetch policies under realistic network contention.

FloatBarrier

10 Related Work

SpectrumKV sits at the intersection of PD disaggregation, KV compression, and KV quantization. We position it relative to each.

LLM serving and PD disaggregation. PagedAttention introduced efficient KV memory management for high-throughput LLM serving [7]. Splitwise, DistServe, and Mooncake split prefill and decode or build KV-centric disaggregated serving systems [15;16;21]. FlowKV and KVServe further target KV transfer latency and communication-efficient disaggregated serving [8;10]. LMCache provides vLLM-integrated KV offloading [12], and NVIDIA Dynamo implements KVBM multi-level KV management for production deployments [14]. These systems treat all transferred KV at uniform precision; SpectrumKV adds per-token precision differentiation and a cold-tier upgrade mechanism that reduces on-demand fetch costs from $s_{16} = 2$ to $s_{16} - s_4 = 1.75$ bytes per element.

KV selection and compression. StreamingLLM identifies attention sinks [17]. H_2O, SnapKV, PyramidKV, and DynamicKV select or compress KV according to heavy-hitter or task-aware signals [1;9;20;22]. These methods make binary keep/drop decisions per token; SpectrumKV retains the importance idea but replaces the drop action with lower-precision transmission, preserving approximate information for every token rather than fully deleting the unselected ones.

KV reuse and lossless compression. CacheBlend enables KV reuse across shared prefixes in RAG workloads by fusing cached KV blocks with newly computed ones [18]. SplitZip applies lossless BF16 compression to KV transfer payloads, reducing network traffic without any quality

loss[5]. Both are orthogonal to SpectrumKV: CacheBlend operates at the prefix level (not per-token), and SplitZip is lossless (not mixed-precision). Combining SplitZip's lossless compression with SpectrumKV's per-token precision allocation could yield further bandwidth savings.

Per-layer KV placement. OrbitFlow[13] formulates KV cache placement as a per-layer integer linear program (ILP) within a real vLLM system, optimizing the assignment of KV layers to GPU, CPU, and disk tiers. SpectrumKV assigns precision *per token* (within a layer), while OrbitFlow assigns storage *per layer* (across devices). The two dimensions are orthogonal and could be combined: a per-layer placement system could use SpectrumKV to reduce each layer's KV payload size before placement.

KV quantization. KVQuant applies per-channel quantization with calibration data to reduce KV memory and bandwidth[6]; KIVI partitions keys and values into separate quantization groups per channel[11]. Both apply uniform precision to all tokens within each group, regardless of token importance; SpectrumKV uses per-token mixed precision and adds an empirical INT4 tolerance probe because aggressive KV quantization can be model-specific. The two approaches are complementary: SpectrumKV assigns per-token tiers, and each tier can use a uniform quantization method such as KIVI (see Section 8).

Model quantization. QuiP[2] and GPTQ[4] quantize model weights rather than KV caches. These methods reduce inference compute and memory but do not address the PD transfer problem: a quantized model still produces a full-precision KV cache during prefill. SpectrumKV operates on the KV cache after it is produced, regardless of the model's weight precision. The two can be composed: a GPTQ-quantized model can still benefit from SpectrumKV for PD transfer.

Working-set view. Denning's working-set model describes the active memory footprint of a process[3], and PagedAttention[7] adapts OS virtual-memory paging to KV block management. SpectrumKV extends this OS–KV analogy to the precision dimension: instead of the binary present/absent distinction of pages, each KV token can occupy an intermediate precision state. Cold-tier fetch in SpectrumKV is analogous to demand paging, but cheaper—upgrading from INT4 to FP16 costs only $s_{16} - s_4 = 1.75$ bytes per element rather than the full $s_{16} = 2$ bytes required to retransmit a previously dropped token.

FloatBarrier

11 Conclusion

PD disaggregation turns the KV cache into a communication object, and existing keep/drop methods reduce its transfer volume at the cost of losing all information from dropped tokens. SpectrumKV replaces that binary decision with a per-token precision spectrum. It protects high-importance tokens at FP16, sends medium-importance tokens at INT8, and uses INT4 for low-importance tokens only when a lightweight probe says the model tolerates it.

The data supports three claims. First, mixed precision preserves PPL substantially better than binary selection at the same budget: at $b = 0.5$, SpectrumKV is within about two PPL-percent on Qwen and is effectively neutral on Mistral and Gemma, while PDTrim degrades by 22–36%. Second, retaining all tokens at some precision preserves retrieval: at $b = 0.3$, adaptive SpectrumKV more than doubles Qwen NIAH accuracy over PDTrim, and Mistral/Gemma preserve retrieval under the 3-tier policy. Third, INT4 tolerance is model-dependent and cannot be inferred from cosine similarity alone; empirical probing is necessary for safe deployment.

The ablation study shows that sink protection provides small but consistent improvements, importance ranking is beneficial at moderate budgets but less critical than expected at low

budgets (where INT4 placement dominates), and the INT4 probe is the single most important safety mechanism for INT4-intolerant models. The theoretical analysis (Lemma 2, Proposition 2) formalizes the softmax amplification mechanism that explains Qwen's INT4 failure and provides error propagation bounds across layers.

The next step is a full PD serving implementation with vLLM-style scheduling, networked KV transfer, cold-tier fetches, and workload-level latency/goodput evaluation.

References

[1] Zefan Cai et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

[2] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, 2023.

[3] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[4] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[5] Yipin Guo and Siddharth Joshi. SplitZip: Ultra fast lossless KV compression for disaggregated LLM serving. *arXiv preprint arXiv:2605.01708*, 2025.

[6] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *Advances in Neural Information Processing Systems*, 2024.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023.

[8] Yichao Li et al. Flowkv: A disaggregated inference framework with low-latency kv cache transfer and load-aware scheduling. *arXiv preprint arXiv:2504.03775*, 2025.

[9] Yuhong Li et al. Snapkv: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

[10] Huan Liu et al. KVServe: Service-aware kv cache compression for communication-efficient disaggregated LLM serving. *arXiv preprint arXiv:2605.13734*, 2026.

[11] Zirui Liu, Jiayi Zhao, Wenxuan Yao, Shijie Cheng, Yucheng Li, Zhe Liang, Jiarong Luo, Xiaodong Wang, Quanquan Gu, Xuehai Li, Yuxuan Liu, Yuandong Wang, and Beidi Chen. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *International Conference on Machine Learning*, 2024.

[12] LMCache Team. LMCache: KV cache offloading for vLLM. *arXiv preprint arXiv:2504.02257*, 2025.

[13] Xinyu Ma et al. OrbitFlow: SLO-aware long-context LLM serving with fine-grained KV cache reconfiguration. *Proceedings of the VLDB Endowment*, 19:1046–1060, 2026.

[14] NVIDIA. NVIDIA Dynamo: Multi-level KV cache management for LLM serving. *NVIDIA Technical Blog*, 2025. `https://developer.nvidia.com/blog/tag/dynamo/`.

[15] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proceedings of the International Symposium on Computer Architecture*, 2024.

[16] Rui Qin, Zhaozhuo Li, Weiran He, Ming Zhang, Yongwei Wu, Weimin Zheng, and Xiaowen Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.

[17] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.

[18] Yihong Yao et al. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.

[19] Hao Zhang, Mengsi Lyu, Yulong Ao, and Yonghua Lin. Enhancing llm efficiency: Targeted pruning for prefill-decode disaggregation in inference. *arXiv preprint arXiv:2509.04467*, 2025.

[20] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

[21] Yinmin Zhong, Shengyu Liu, Junda Chen, Jian Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*, 2024.

[22] Yuxuan Zhou et al. Dynamickv: Task-aware adaptive kv cache compression for long context llms. *arXiv preprint arXiv:2412.14838*, 2024.

A Decay-Rate Sweep

The decay-rate sweep in Table 15 comes from the simulation-v3 scan rather than the main GPU PPL suite. It is included as an auxiliary sensitivity result. Larger decay rates help Mistral and Gemma in this sweep, while Qwen is less informative because it uses a KV-norm fallback.

Table 15: Decay-rate sweep.

Model	Decay rate	$b = 0.3$	$b = 0.5$	$b = 0.7$
Mistral (attn×decay)	0.001	+36.7	+11.7	+1.9
	0.005	+27.5	+0.6	−10.7
	0.010	+24.7	−2.0	−12.3
Gemma (attn×decay)	0.001	+14.1	−0.2	−5.3
	0.005	+13.9	−12.7	−15.2
	0.010	+13.4	−13.2	−18.1
Qwen (KV-norm×decay)	0.001	+38.4	+9.6	−2.0
	0.005	+44.5	+12.5	−1.3
	0.010	+45.7	+12.5	−1.3

B Experiment Log Notes

The final data was produced after we corrected several implementation bugs. The most important were: (i) replacing SDPA hook measurements with eager attention for locality characterization; (ii) replacing zero-out baselines with attention-mask deletion for PDTrim and SWS; (iii) adding the missing NIAH retrieval question; (iv) fixing an if/elif interaction that made adaptive 2-tier results too optimistic by leaving some dropped positions visible; and (v) applying the Gemma chat template. These notes are included to make the evidence boundary explicit.

C Raw Numerical Data

This appendix provides the absolute PPL values, TTFT/TPS standard deviations, and per-layer cosine similarities underlying the main-text tables. Reporting these values enables direct comparison and meta-analysis by other researchers.

C.1 Absolute PPL values at $b = 0.5$ (Table 2)

The main text reports PPL as percentage change from the FP16 baseline. Table 16 gives the absolute PPL values.

Table 16: Absolute PPL at $b = 0.5$, seq=2048. Baseline FP16 values in parentheses.

Method	Qwen2.5-7B (7.0415)	Mistral-7B (7.9568)	Gemma-2-9B (11.2042)
$\text{Uniform}_{\text{INT8}}$	7.1805	7.9517	11.1548
PDTrim	8.8614	9.7132	15.1960
$\text{SWS}_{\text{Original}}$	9.2159	10.6025	16.1092
$\text{SWS}_{\text{ValueAware}}$	9.2159	10.6025	16.1092
$\text{SpectrumKV}_{\text{Greedy}}$	7.1805	7.9517	11.1548
$\text{SpectrumKV}_{\text{SinkProtect}}$	7.1397	7.9492	11.1514
$\text{SpectrumKV}_{\text{Balanced}}$	535.6371	8.1721	11.4623
$\text{Random}_{\text{Tier}}$	7.1805	7.9517	11.1548

C.2 Budget sweep absolute PPL (Table 3)

Table 17 provides the absolute PPL values for the budget sweep.

Table 17: Absolute PPL across budgets at seq=2048.

Model	Method	$b = 0.3$	$b = 0.5$	$b = 0.7$
Qwen2.5-7B	PDTrim	10.0537	8.8614	8.3188
	SWS	11.0382	9.2159	8.1492
	SpectrumKV Greedy	unsafe	7.1805	7.0746
Mistral-7B	PDTrim	10.4199	9.7132	9.0991
	SWS	11.7971	10.6025	9.5055
	SpectrumKV Greedy	8.3874	7.9517	7.9592
Gemma-2-9B	PDTrim	20.3668	15.1960	12.9208
	SWS	22.5519	16.1092	13.7217
	SpectrumKV Greedy	11.4790	11.1548	11.2008

C.3 Context-length sweep absolute PPL (Table 6)

Table 18 shows absolute PPL for the context-length experiment at $b = 0.5$.

Table 18: Context-length PPL at $b = 0.5$.

Model	Seq	FP16 PPL	SpectrumKV Greedy	SpectrumKV SinkProtect	PDTrim
Qwen2.5-7B	2048	7.0415	7.1805	7.1397	8.8614
Qwen2.5-7B	4096	6.0084	6.1557	6.1141	6.9986
Qwen2.5-7B	8192	6.2145	6.3949	6.3721	6.6645
Mistral-7B	2048	7.9568	7.9517	7.9492	9.7132
Mistral-7B	4096	6.2379	6.2296	6.2397	7.0924
Mistral-7B	8192	6.4185	6.4000	6.4136	6.9282
Gemma-2-9B	2048	11.2042	11.1548	11.1514	15.1960
Gemma-2-9B	4096	7.9672	7.9784	7.9513	8.7090
Gemma-2-9B	8192	—	—	—	—

C.4 Per-layer INT4 cosine similarity (Table 8)

Tables 19–21 list the per-layer INT4 key and value cosine similarity for all three models.

Table 19: Per-layer INT4 cosine similarity for Qwen2.5-7B (28 layers).

Layer	Key cos	Val cos	Layer	Key cos	Val cos
0	0.9947	0.9816	14	0.9811	0.9892
1	0.9888	0.9873	15	0.9747	0.9884
2	0.9756	0.9851	16	0.9805	0.9908
3	0.9718	0.9897	17	0.9838	0.9910
4	0.9780	0.9808	18	0.9775	0.9915
5	0.9699	0.9860	19	0.9689	0.9911
6	0.9747	0.9871	20	0.9828	0.9916
7	0.9712	0.9922	21	0.9740	0.9922
8	0.9776	0.9864	22	0.9796	0.9924
9	0.9741	0.9928	23	0.9745	0.9909
10	0.9706	0.9903	24	0.9684	0.9915
11	0.9747	0.9874	25	0.9706	0.9915
12	0.9809	0.9909	26	0.9750	0.9928
13	0.9678	0.9911	27	0.9955	0.9925

Table 20: Per-layer INT4 cosine similarity for Mistral-7B (36 layers).

Layer	Key cos	Val cos	Layer	Key cos	Val cos
0	0.9896	0.9804	18	0.9806	0.9921
1	0.9916	0.9855	19	0.9789	0.9915
2	0.9758	0.9870	20	0.9782	0.9913
3	0.9825	0.9896	21	0.9765	0.9912
4	0.9790	0.9906	22	0.9797	0.9906
5	0.9789	0.9900	23	0.9759	0.9914
6	0.9771	0.9898	24	0.9763	0.9906
7	0.9804	0.9903	25	0.9733	0.9924
8	0.9772	0.9926	26	0.9744	0.9931
9	0.9756	0.9898	27	0.9785	0.9922
10	0.9809	0.9913	28	0.9757	0.9914
11	0.9800	0.9885	29	0.9787	0.9927
12	0.9796	0.9910	30	0.9741	0.9885
13	0.9796	0.9893	31	0.9795	0.9925
14	0.9734	0.9899	32	0.9769	0.9909
15	0.9823	0.9913	33	0.9802	0.9928
16	0.9804	0.9910	34	0.9801	0.9929
17	0.9776	0.9929	35	0.9912	0.9931

Table 21: Per-layer INT4 cosine similarity for Gemma-2-9B (42 layers).

Layer	Key cos	Val cos	Layer	Key cos	Val cos
0	0.9658	0.9750	21	0.9746	0.9869
1	0.9334	0.9836	22	0.9760	0.9828
2	0.9476	0.9838	23	0.9751	0.9852
3	0.9416	0.9865	24	0.9771	0.9850
4	0.9532	0.9874	25	0.9740	0.9826
5	0.9535	0.9821	26	0.9734	0.9860
6	0.9600	0.9847	27	0.9597	0.9764
7	0.9487	0.9849	28	0.9744	0.9874
8	0.9631	0.9867	29	0.9763	0.9797
9	0.9592	0.9879	30	0.9723	0.9837
10	0.9581	0.9871	31	0.9666	0.9763
11	0.9469	0.9868	32	0.9700	0.9858
12	0.9730	0.9875	33	0.9565	0.9864
13	0.9627	0.9848	34	0.9685	0.9884
14	0.9628	0.9881	35	0.9574	0.9838
15	0.9680	0.9839	36	0.9589	0.9747
16	0.9683	0.9851	37	0.9613	0.9866
17	0.9728	0.9886	38	0.9593	0.9842
18	0.9798	0.9849	39	0.9619	0.9820
19	0.9672	0.9841	40	0.9555	0.9836
20	0.9747	0.9861	41	0.9623	0.9883